

Sumário

Introdução	2
Transformers	2
Attention is All You Need	2
1 Arquitetura do Modelo	2
1.1 Pilhas de Encoder e Decoder	3
1.2 Attention	4
1.2.1 Aplicações do Attention no Modelo	4
1.3 Position-wise Feed-Forward Networks	5
Foundation Models	5
A Comprehensive Review of Biosignal Foundation Models	5
1 Modelos de Fundação	5
1.1 Definição de Modelos de Fundação	5
1.2 Arquiteturas de Modelo	6
1.2.1 Convoluções e Encoding Preliminar	6
1.2.2 Transformers	6
1.2.3 Vector Quantisation	6
1.2.4 State-Space Models (Mamba)	6
1.3 Arquiteturas e Objetivos SSL de modelos conhecidos	7
1.4 Objetivos de Treino Auto-Supervisionado	7
1.5 Métodos de Avaliação	8

Introdução

Notas pessoais sobre os artigos. A intenção é entender a teoria completa por trás de Foundation Models, para entender como implementar e avaliar.

A intenção é ser enxuto para posterior consulta rápida.

Transformers

Attention is All You Need

Nota rápida: Esse artigo juntou várias pesquisas para criar um modelo final.

- Attention Mechanisms - “Neural Machine Translation by Jointly Learning to Align and Translate”, “Effective Approaches to Attention-based Neural Machine Translation”
- Arquitetura Encoder-Decoder (Seq2Seq) - “Sequence to Sequence Learning with Neural Networks”, Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation“
- Conexões residuais (blocos Add) - “Deep Residual Learning for Image Recognition”
- Normalização de Camada - “Layer Normalization”
- Positional Embeddings - “End-To-End Memory Networks”, “Convolutional Sequence to Sequence Learning”
- Multi-Head Self-Attention - “A Structured Self-Attentive Sentence Embedding”

1 Arquitetura do Modelo

A maioria dos modelos competitivos de transdução (conversão) de sequência neural possuem uma estrutura encoder-decoder.

- **Encoder** - Mapeia uma sequência de representação simbólica $(x_1, \dots, x_n)$ para uma sequência de representação contínua $z = (z_1, \dots, z_n)$, **vetores que capturam o significado e o contexto de cada input**.
- **Decoder** - Dado z , gera uma sequência output $(y_1, \dots, y_m)$ de símbolos um elemento por vez.

A cada passo o modelo é auto-regressivo, consumindo os símbolos gerados previamente como input adicional enquanto gera o próximo.

(Ex: tradução automática - Pega palavras, caputra significado, gera output em outra língua)

O Transformer segue essa arquitetura geral usando self-attention empilhados e processamento de um único item/elemento por vez.

1.1 Pilhas de Encoder e Decoder

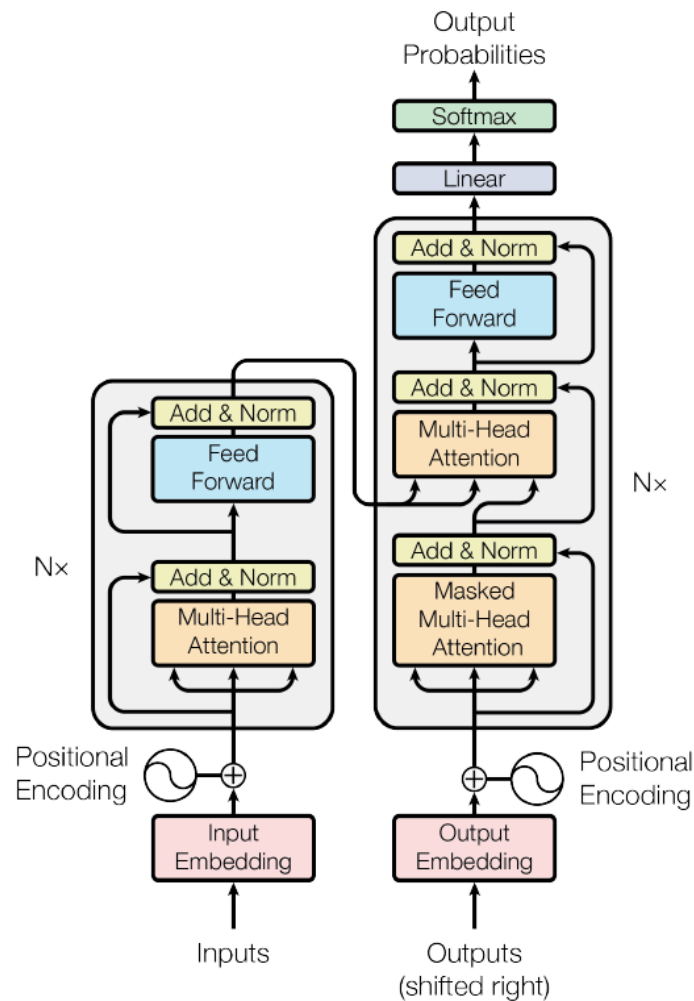


Figure 1: The Transformer - model architecture.

Encoder

- Composto de uma pilha de 6 camadas idênticas com 2 Sub-Camadas cada.
- Camadas:
 1. Mecanismo **Multi-head Self-Attention** - Múltiplos cabeçotes atuando em paralelo. Cada um aprende a focar em aspectos/relações diferentes entre as palavras. No final, os resultados são concatenados.
 2. **Position-wise Fully Connected Feed-Forward Network (FFN)** - Rede neural densa (FFN) aplicada de forma independente e idêntica a cada posição/token da sequência.
- **Output Sub-Camada:** $\text{LayerNorm}(x + \text{Sublayer}(x))$ (Add + Normaliza)
- Todas as Sub-Camadas possuem dimensionalidade $d = 512$

O **Self-Attention decide onde olhar** (decide quais palavras são importantes) e **Position-Wise FFN processa e transforma o output em conceitos abstratos** (relações sintáticas complexas, papéis semânticos e tipos de entidades). É um **vetor enriquecido**.

Decoder

- Também composto de uma pilha de 6 camadas idênticas, com 3 Sub-Camadas cada

- Camadas:
 1. **Masked Multi-Head Attention** - Mecanismo Self-Attention modificado para prevenir posições de prestar atenção a posições subsequentes (posteriores). Força o modelo a ignorar o que vem depois. (se não tivesse isso, no treinamento o modelo “colaria” a resposta do futuro, ao invés de tentar aprendê-la)
 2. **Multi-Head Attention** - Atua sobre o output da pilha do Encoder
 3. **Position-wise Fully Connected Feed-Forward Network (FFN)** - Consolida a mistura (passado gerado + contexto do encoder) e faz a predição.

Na entrada do decoder, temos o **histórico de tokens já gerados**.

Linear

Transforma o vetor de dimensão d_{model} para um vetor de dimensão d_{vocab} .

Softmax

Gera as probabilidades de cada palavra do vocabulário ser o próximo token.

1.2 Attention

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q = Matriz Query (dimensão d_k)
- K = Matriz Key (dimensão d_k)
- V = Matriz Value (dimensão d_v)
- $\sqrt{d_k}$ = Normalização

Multi-Head Attention

Permite ao modelo prestar atenção a informações de diferentes subespaços de representação em diferentes posições. Só um cabeçote de atenção inibe isso (média).

As Queries/Keys/Values são projetados linearmente h vezes com diferentes projeções lineares aprendíveis. Os diferentes outputs são processados em paralelo, concatenados e projetados novamente.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- Projeções são matrizes de parâmetros $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$

No paper, $h = 8$ camadas de atenção paralelas. $d_k = d_v = \frac{d_{\text{model}}}{h} = 64$. (Assim, para um input de 512 itens, cada cabeçote vai analisar um recorte do input, com 64 itens)

1.2.1 Aplicações do Attention no Modelo

- Em camadas “encoder-decoder attention”, Q (queries) vem do output do **decoder** anterior, e K/V (Key, Values) vêm do output do **encoder**. Isso permite que toda posição no decoder

preste atenção a todas as posições da sequência de input. Ou seja, a representação inteira do texto de entrada (processada pelo Encoder) fica disponível como um mapa de referência (V,K). A camada anterior do decoder diz: “Estou tentando gerar a próxima palavra... O que preciso buscar no texto original?”

- O encoder contém camadas self-attention. Em uma camada, K,V,Q vêm do mesmo lugar: a saída da camada anterior. Cada posição pode prestar atenção a todas as posições na camada anterior.
- No decoder, da mesma forma. Previne-se vazamento de informação a ser adivinhada com uma máscara, definindo para $-\infty$ tudo que não se quer.

1.3 Position-wise Feed-Forward Networks

A FFN é aplicada a cada posição separadamente e identicamente. Consiste em duas transformações lineares com uma ativação ReLU entre elas.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

(outra maneira de descrever isso é como duas convoluções de Kernel size 1).

Foundation Models

A Comprehensive Review of Biosignal Foundation Models

1 Modelos de Fundação

1.1 Definição de Modelos de Fundação

Definição: Modelo Grande pré-treinado em dados de larga-escala para aprender representações ricas em informação que podem ser usadas para tarefas específicas. **Ao invés de modelos de deep-learning, que são específicos para uma tarefa, um modelo de fundação busca capturar representações universais entre domínios e/ou modalidades**, servindo como um modelo de propósito geral.

Um modelo de fundação aprende uma função:

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Z}$$

- $\mathcal{X}$ = Dataset
- $\mathcal{Z}$ = Espaço Latente
- θ = Parâmetros Treináveis

O objetivo de um modelo de fundação é encontrar os parâmetros que minimizam a função Loss:

$$\mathcal{L}(f_{\theta}(x), t(x))$$

- $t(x)$ = Objetivo Auto-Supervisionado (Ex: Masked Prediction, Contrastive Learning Prediction, Next-Step Prediction). É exatamente o que o modelo deveria prever.

1.2 Arquiteturas de Modelo

Transformers/Mamba são o esqueleto principal do Modelo de Fundação. Eles são precedidos/antecedidos de módulos menores.

1.2.1 Convoluções e Encoding Preliminar

Convoluções sobre a dimensão do tempo são normalmente aplicadas.

1.2.2 Transformers

Seu mecanismo-chave é o self-attention, que **permite ao modelo considerar relações entre todos os elementos/tokens da sequência de input.**

$$H = \text{Attention}(XQ, XK, XV) = \text{Softmax}\left(\frac{XQ(XK)^T}{\sqrt{d_k}}\right)$$

- H = Representação final
- X = Sequência de tokens $[x_1, \dots, x_T]$
- Q, K, V = Matrizes de projeção aprendíveis
- d_k = Dimensionalidade chave (regulariza valores)

O Transformer é construído com blocos encoder/decoder empilhados.

- Blocos Encoder - Contém módulos **multi-head self-attention** e **position-wise feedforward networks**
- Blocos Decoder - Incorporam **cross-attention** para os outputs do encoder.

Modelos de fundação podem usar a estrutura encoder/decoder completa ou não.

- BERT style - Apenas Encoder
- GPT style - Apenas Decoder

Transformers possuem complexidade $\mathcal{O}(T^2)$.

Adicional: Na última camada, o Transformer gera uma **distribuição de probabilidades sobre o vocabulário para determinar o próximo token.**

1.2.3 Vector Quantisation

Em alguns casos, modelos EEG são pré-treinados em dois estágios:

1. **Tokenizer é treinado** para gerar tokens significativos das amostras de input (que representem bem o input)
2. Tokenizer é usado para treinar Modelo de Fundação

Em todos os modelos de EEG de dois estágios, o tokenizador contém um codebook de quantização vetorial (VQ).

Depois que a entrada for codificada em alguma representação latente h , um livro de códigos aprendível $V = \{v_i, \dots, v_K\}$ é usado para buscar o código mais próximo de h .

1.2.4 State-Space Models (Mamba)

Alternativa aos transformers.

$$h'(t) = Ah(t) + Bx(t), y(t) = Ch(t)$$

- $x(t)$ = Input
- $h(t)$ = Estado oculto
- $y(t)$ = Output no tempo t
- A, B, C = Parâmetros aprendíveis

Na prática, isso é discretizado.

1.3 Arquiteturas e Objetivos SSL de modelos conhecidos

Model	Architecture	SSL Objective	Model Size	Open source
ALFEE [58]	Temporal CNN, Spatial Transformer, Temporal Transformer, Decoder Transformer	Masked patch reconstruction, Signal forecasting	16.3M-540M	No
BENDR [49]	wav2vec CNN, Transformer	Contrastive learning	33M	Yes
BioSerenity-E1 [53]	1st stage: Temporal CNN, VQ Codebook, Decoder; 2nd stage: Temporal CNN, Transformer	1st stage: DPSS reconstruction, Codebook loss; 2nd stage: Masked code prediction	12.6M	No
BIOT [22]	FC Tokenizer, Transformer	Masked embedding reconstruction	3.2M	Yes
BrainBERT [23]	Transformer	Masked STFT/superlet reconstruction	43.2M	Yes
BrainGPT [50]	Temporal Transformer with causal attention	Autoregressive next patch prediction	1.5M-1.1B	No
Brant [24]	Temporal Transformer, Spatial Transformer	Masked patch reconstruction	505.7M	No
Brant-2 [62]	Temporal and Frequency CNNs, Temporal Transformer, Spatial Transformer	Masked patch reconstruction, Forecasting	1B	No
CBraMod [48]	Spatio-Temporal Transformer with "Criss-Cross Attention"	Masked patch reconstruction	4M	Yes
CEReBrO [55]	Alternating Spatial and Temporal Transformer	Masked token reconstruction	3.6M-85M	No
CodeBrain [63]	1st stage: CNN, Transformer, Time and Frequency VQ Codebooks, Time and Frequency Transformers; 2nd stage: Temporal CNN, Convolutional SSM, Sliding Window Attention	1st stage: Reconstruction (patch, FFT amplitude and phase), Codebook loss; 2nd stage: Masked code prediction	16M	No
CSBrain [64]	Multi-scale Temporal CNN, Multi-scale Spatial CNN, Spatio-Temporal Transformer	Masked patch reconstruction	1.4M-4.9M	No
DIVER-0 [59]	Temporal CNN, Transformer	Masked patch reconstruction	-	No
EEGFormer [51]	Temporal Transformer, VQ Codebook, Transformer	Masked patch reconstruction, Codebook loss	-	No
EEGM2 [42]	Multi-scale Temporal CNN, Encoder Mamba-2, Mediator Mamba-2, Decoder Mamba-2	Signal reconstruction, rFFT reconstruction	0.3M-4.5M	Yes
EEGMamba [60]	Temporal CNN, Alternating forward/backward Mamba-2 blocks	Masked patch reconstruction	3.3M	Yes
EEGPT [46]	Encoder Transformer, Predictor Transformer, Reconstructor Transformer; Momentum Encoder Transformer	Masked patch reconstruction	4.7M-25M	Yes
EEGTrans [43]	1st stage: Temporal CNN, RVQ Codebook, CNN Decoder; 2nd stage: Transformer, CNN Decoder	1st stage: Reconstruction (signal, mean and std), Codebook loss; 2nd stage: signal reconstruction, next code prediction	-	Yes
FEMBA [65]	2D CNN, Mamba with forward and backward branches, CNN Decoder	Masked patch reconstruction	745M	No
FoME [44]	Temporal Transformer, Spatial Transformer	Masked patch reconstruction	-	Yes
GEFM [66]	GNN, wav2vec CNN, Transformer	Contrastive learning	-	No
LaBraM [52]	1st stage: Temporal CNN, VQ Codebook, Transformer; 2nd stage: Temporal CNN, Transformer	1st stage: Reconstruction (FFT amplitude and phase), Codebook loss; 2nd stage: Masked code prediction	5.8M-369M	Yes
LCM [67]	CNN, Transformer	Masked patch reconstruction	-	No
LEAD [54]	Temporal Transformer and Spatial Transformer	Contrastive learning	33.9M	Yes
M4CEA [68]	Temporal CNN, Transformer	Masked reconstruction (FFT amplitude and phase)	5.84M	Yes
MENDR [47]	GAT, 2D CNN, Transformer	Wavelet reconstruction	1.8M-6.5M	No
Neuro-GPT [45]	Temporal CNN, Transformer	Autoregressive next token prediction	78.5M	Yes
NeuroLM [69]	1st stage: Temporal CNN, Transformer, VQ Codebook; 2nd stage: Pre-trained LLM	1st stage: Reconstruction (signal and FFT amplitude), Codebook loss 2nd stage: Autoregressive next code prediction	254M-1.7B	Yes
SS-EMERGE [56]	Temporal GAT and Feature GAT, CNN	Contrastive learning	-	Yes

TABLE 2
Summary of Foundation Models trained on EEG data including scalp EEG and intracranial EEG (iEEG).

1.4 Objetivos de Treino Auto-Supervisionado

- **Masked Autoencoding** - Máscara aplicada aos Dados ou Espaço Latente, e o modelo é treinado para reconstruir porções mascaradas. **Reconstrução de componentes faltantes a partir de componentes visíveis.** A Loss deve medir a fidelidade da reconstrução.

- **Autoregressive Models** - Tenta adivinhar o próximo token/segmento na sequência. Usa a Cross-Entropy Loss.
- **Contrastive Learning** - Tenta identificar relações de similaridade entre pares positivos/negativos no espaço latente $\mathcal{Z}$. Encoraja minimizar a distância entre pares positivos e maximizar a distância entre pares negativos. “Estas duas imagens são variações do mesmo objeto ou são objetos diferentes?”

1.5 Métodos de Avaliação

Modelos de Fundação são avaliados avaliando sua performance em tarefas downstream como: Classificação, Regressão, Previsão, Imputação.

O modelo pode ser adaptado com:

- **Full-finetuning**: Todos os pesos são levemente treinados com dados downstream.
- **Linear Probing**: Modelo congelado, adiciona-se cabeça de classificação treinável

Para Previsão/Imputação, pode ser usado **Zero-shot Learning**: Modelo usado diretamente em dados nunca vistos.

Métricas de avaliação comuns:

- **Acurácia** - Proporção de acertos totais (positivos/negativos) em relação ao total de previsões.
- **Acurácia Balanceada** - Acurácia obtida em cada classe individualmente
- **AUPRC (Area Under Precision-Recall Curve)** - Mede área abaixo da curva que cruza a Precisão (ver quantos dos previstos como positivos eram realmente positivos) e o Recall (quantos dos positivos reais o modelo encontrou) em diferentes limiares.
- **AUROC (Area Under the Receiver Operating Characteristic Curve)** - Mede capacidade do modelo de separar/distinguir classes. Plota a taxa de verdadeiros positivos x falsos positivos.
- **F1-Score** - Média harmônica entre Precisão e Recall
- **MSE (Mean Squared Error)** - Para tarefas de regressão